

1. como eu posso fazer esse trabalho de acordo com o pdf e o arquivo do professor?
nao sei como comecar

Resposta:

O melhor jeito é começar olhando primeiro o que já veio pronto no arquivo do professor, sem sair apagando ou criando coisas sem necessidade. Pelo PDF, o trabalho precisa ser uma API em Node.js com Express, usando SQLite com Knex, migrations, seeders, JWT, arquitetura em camadas e consumo de APIs externas.

Então eu começaria por partes:

Primeiro, conferir a estrutura inicial do projeto e manter os arquivos base do professor, como `server.js`, `package.json`, `knexfile.js` e a configuração do banco.

Depois, criar as migrations pelo comando do Knex, porque o banco não deve ser criado manualmente. As migrations precisam criar as tabelas `usuarios` e `palpites`.

Em seguida, criar um seeder para inserir dois usuários de teste. Assim o professor consegue rodar o projeto e testar o login logo depois de clonar.

Depois disso, implementar o login com JWT. A rota `POST /api/login` deve receber email e senha, buscar o usuário no banco e, se estiver certo, devolver um token com o id do usuário.

Depois, criar um middleware para proteger as rotas privadas. Esse middleware vai ler o token no header `Authorization`, validar o JWT e salvar o id do usuário em `req.usuarioId`.

Depois disso, implementar as rotas de palpites: criar, listar, atualizar e deletar. Todas essas rotas devem usar o id do usuário que vem do token, não do body ou da URL.

Na criação do palpite, também precisa usar `Promise.all` com `axios` para buscar o valor do dólar e os feriados em APIs externas. Depois esses dados devem ser salvos junto com o palpite no banco.

Por último, criar a rota pública `GET /api/jogos`, que busca os jogos em uma API externa e devolve uma resposta tratada, sem mandar o JSON cru inteiro.

O ideal é fazer e testar uma parte por vez no Postman: primeiro login, depois rota protegida sem token, depois criar palpite, listar palpites, atualizar, deletar e por fim listar jogos.

Comentário:

Entendi que dava pra fazer em etapa sem parecer um bicho de sete cabeças. Primeiro banco com migrations, depois seeders, login, autenticação, depois CRUD de palpites e por último as APIs externas.

2. como crio migrations

Resposta:

O correto é criar as migrations pelo comando do Knex. Tu não precisa criar o arquivo manualmente com o nome completo.

No terminal, dentro da pasta do projeto, roda:

```
npx knex --knexfile src/database/knexfile.js migrate:make create_usuarios
```

Depois roda:

```
npx knex --knexfile src/database/knexfile.js migrate:make create_palpites
```

O Knex vai criar os arquivos automaticamente dentro da pasta de migrations. Depois que ele criar, tu abre cada arquivo e coloca a estrutura da tabela dentro dele.

Comentário:

dessa forma aprendi o comando certinho para criar as pastas do migrations, do modo automático sem ser o manual.

3. dentro deles ta cheio de coisa dentro. posso apagar oq tem la e colocar o codigo?

Resposta:

Sim, pode apagar o conteúdo que veio dentro do arquivo de migration, porque aquilo é só o modelo automático do Knex. O importante é não apagar a pasta e não apagar arquivos originais do professor.

Nos arquivos novos de migration, tu pode substituir o conteúdo pelo código que cria a tabela. No arquivo `create_usuarios`, coloca os campos da tabela de usuários. No arquivo `create_palpites`, coloca os campos da tabela de palpites.

Comentário:

Entendi que os comentários e o modelo que aparecem dentro dos arquivos criados são gerados automaticamente pelo Knex. Então eu podia apagar esse modelo e colocar a criação da tabela que não teria problema.

4. como coloco biblioteca de hash

Resposta:

Para colocar uma biblioteca de hash no projeto, podemos usar o `bcryptjs`, que serve para transformar a senha em um hash antes de salvar no banco.

No terminal, dentro da pasta do projeto, roda:

`npm install bcryptjs`

Depois, no arquivo onde tu precisa gerar o hash, importa a biblioteca assim:

```
const bcrypt = require('bcryptjs');
```

Para criar o hash da senha, usa:

```
const senhaCriptografada = await bcrypt.hash('123456', 10);
```

Esse número **10** é o nível de salt, que ajuda a deixar o hash mais seguro.

Depois, quando for fazer login, tu não compara a senha digitada diretamente com a senha salva no banco. Tu usa:

```
const senhaValida = await bcrypt.compare(senhaDigitada, usuario.senha);
```

Se **senhaValida** for **true**, significa que a senha digitada corresponde ao hash salvo no banco.

Comentário:

Entendi que a biblioteca de hash serve para não salvar a senha em texto puro no banco. No caso tem a senha real mas aí cria um código que camufla ele e aí fica criptografado.

Também entendi que no login usa o `bcrypt.compare` para verificar se a senha digitada bate com o hash salvo.

5. ta mas como faco isso de login e gerar token

Resposta:

Tu precisa criar uma rota pública **POST /api/login**. Essa rota recebe **email** e **senha** no body.

No service, primeiro busca o usuário pelo email na tabela **usuarios**. Se não encontrar, retorna erro 401. Se encontrar, compara a senha digitada com o hash salvo no banco usando **bcrypt.compare**.

Se a senha estiver correta, gera um token com JWT:

```
jwt.sign({ id: usuario.id }, JWT_SECRET, { expiresIn: '1d' })
```

Esse token deve ser retornado na resposta. O id do usuário vai dentro do token para ser usado depois nas rotas protegidas.

Comentário:

Entendi que precisa criar a rota pública e aí ele verifica se o usuário existe e gera o token.

6. como faco o negocio de promisse

Resposta:

No service de criar palpite, tu usa `Promise.all` com duas chamadas axios ao mesmo tempo.

Uma chamada busca o dólar na AwesomeAPI:

```
axios.get('https://economia.awesomeapi.com.br/json/last/USD-BRL')
```

A outra busca os feriados do ano da data do jogo na BrasilAPI:

```
axios.get('https://brasilapi.com.br/api/feriados/v1/${ano}')
```

Depois tu pega o valor do dólar na resposta e verifica se a data do jogo existe dentro da lista de feriados. Com isso, salva no banco o palpite, o valor do dólar e se era feriado ou não.

Comentário:

Entendi que o promisse serve para fazer as duas chamadas ao mesmo tempo.

7. ta e como q eu uso o negocio de api que pede

Resposta:

No trabalho, tem dois usos de API externa.

O primeiro é na rota de listar jogos. Nessa parte, tua API precisa buscar jogos em outra API usando `axios.get`. Tu precisa deixar essa API rodando localmente em outro terminal. Depois, no teu backend, cria uma rota pública:

GET /api/jogos

Dentro do service dessa rota, tu usa o axios para buscar os jogos:

```
const resposta = await axios.get('http://localhost:3333/games');
```

Depois tu não devolve a resposta crua inteira. Tu faz um `map` para devolver só as informações principais, como data, etapa, grupo, estádio, time da casa e time de fora.

O segundo uso de API externa é na hora de criar um palpite. Quando o usuário chama:

POST /api/palpites

o sistema precisa buscar duas informações externas ao mesmo tempo: a cotação do dólar e os feriados do ano do jogo. Para isso, usa `Promise.all`:

```
const [respostaDolar, respostaFeriados] = await Promise.all([
  axios.get('https://economia.awesomeapi.com.br/json/last/USD-BRL'),
  axios.get('https://brasilapi.com.br/api/feriados/v1/${ano}')
]);
```

Depois tu pega o valor do dólar na resposta da AwesomeAPI e verifica se a data do jogo existe na lista de feriados da BrasilAPI. Com isso, salva no banco o palpite junto com `dolar_no_dia` e `dia_de_feriado`.

Resumindo: a WorldCupAPI é usada para listar jogos, e a AwesomeAPI junto com a BrasilAPI são usadas na criação do palpite.

Comentário:

Entendi que isso faz o sistema buscar dados em outro usando axios.